

Implementazione dell'algoritmo del simplexso generale

Dimitri B.

21 settembre 2026

1 Introduzione

In questo progetto si riporta l'implementazione dell'algoritmo del simplexso generale in Rust. Si fa utilizzo inoltre del generatore di parser Tree-sitter, la cui grammatica è stata scritta in JavaScript. Il formato dei file di input e output è testuale e il programma si serve di una semplice interfaccia da riga di comando per chiedere all'utente le informazioni necessarie per eseguire e risolvere i sistemi lineari sottoposti. I seguenti libri sono stati usati come riferimento insieme al materiale raccolto a lezione ([1], [2]).

Di seguito riporteremo una spiegazione della struttura dell'input e dell'output, una breve panoramica della grammatica e della struttura del parser, un accenno alle scelte fatte durante lo sviluppo del programma e un riassunto dei diversi test effettuati per verificare le performance del software.

2 Input & Output

L'input deve essere strutturato nel seguente modo. In un sistema lineare, le equazioni devono essere separate dal token `'and'`, una scelta dettata da motivi puramente estetici e di somiglianza con la forma logica. Le variabili e i parametri

possono essere stringhe a più caratteri (per esempio `"x1"` o `"var"`) e, se posti uno accanto all'altro, devono essere separati dal simbolo di moltiplicazione `"*"`.

A causa di limitazioni della grammatica, non è possibile riportare costanti dopo una variabile o un parametro (es. `x2`), dato che verrebbero riconosciute come parte del nome della variabile stessa. Se il sistema presenta dei parametri, è necessario appendere, dopo l'ultima equazione, il token `"and parameters:"` elencando poi i parametri e il valore massimo intero che possono assumere.

L'output riporta la soluzione del sistema lineare all'interno di una cornice divisoria. Nel caso in cui questo sia soddisfacibile, il programma lo specifica e mostra la prima soluzione trovata; in caso negativo, ne riporta l'insoddisfaccibilità. Se sono presenti dei parametri, il programma restituirà una soluzione per ogni possibile combinazione di questi, differenziandole tra loro all'interno della cornice divisoria dedicata a quel sistema.

3 Parser

Per analizzare l'input del programma si utilizza il generatore di parser Tree-sitter. Si è scelto di servirsi di uno

strumento di parsing per facilitare la navigazione dell'input, favorire una più intuitiva estrazione delle informazioni e garantire una possibile espansione futura del programma e della grammatica.

Quest'ultima è concisa e cerca di catturare numerose possibilità di scrittura dell'input, pur possedendo alcune limitazioni. In questa versione, ad esempio, non vengono riconosciute le equazioni contenenti espressioni tra parentesi o elevamenti a potenza. Si tratta di una precisa decisione implementativa, dato che il focus del progetto è l'algoritmo e non la grammatica. Fatta questa premessa, il parser rimane comunque molto flessibile e capace di gestire un'ampia gamma di input, come è osservabile dai numerosi test e dal contenuto dei file di esempio.

4 Struttura del sistema

Il programma si presenta all'utente con un menu conciso nel quale viene richiesto il nome del file di input e l'ordinamento che si vuole utilizzare all'interno dell'algoritmo. Se il processo va a buon fine senza errori, al termine dell'esecuzione verrà generato un file "output.txt" nel quale sarà possibile osservare i risultati calcolati. Dietro a questa veloce esecuzione troviamo alcune scelte implementative che distinguono il programma.

In particolare, è rilevante la gestione dei sistemi lineari parametrici. Ogni parametro è associato a un intero positivo che ne rappresenta il valore massimo assumibile (partendo da 0). Verranno quindi creati tanti sistemi lineari quante sono le combinazioni dei valori possibili.

Si è inoltre scelto di dedicare ogni riga a un diverso sistema lineare, in modo da condensare più sistemi all'interno

di un solo file e permettere la risoluzione di un alto numero di istanze in una sola esecuzione.

Infine, vengono considerati tre diversi tipi di ordinamento delle variabili: normale, inverso e random. Nel primo, le variabili basiche hanno un indice più basso rispetto a quelle non basiche; nel secondo l'ordine viene invertito (quello che precedentemente era $b_1 < b_2 < x_1 < x_2$ diventa $x_2 < x_1 < b_2 < b_1$).

5 Performance

In questa ultima sezione ci focalizzeremo su alcuni esperimenti fatti per valutare la correttezza e le performance generali del programma. Gli esperimenti consistono nell'esecuzione del programma sottoponendo diverse tipologie di input, dove il numero di variabili/equazioni/sistemi è stato modificato per evidenziare un aumento nei tempi di esecuzione ed una differenza tra esecuzione con ordinamento inverso/normale/random.

Senza alcuna sorpresa, per sistemi con un basso numero di equazioni e variabili otteniamo tempi di esecuzione inferiori ai 2ms, tuttavia ai fini di esplorare le potenzialità del programma ci focalizzeremo su casi più complessi ed unici.

Il primo file sarà **Input_base.txt**. In questo caso abbiamo 39 sistemi (copiati da [3]), 35 dei quali parametrici con diversi valori massimi assegnati. Considerando le diverse combinazioni parametriche verranno create e processate 363 istanze dei sistemi lineari differenti, il tutto in circa 7 ms.

Considerando che i tempi riportati sono una media di diverse misurazioni, l'ordinamento che spicca risulta essere quello invertito, che riporta un leggero miglioramento di 6ms sulle alternative.

Tabella 1: Input_base.txt

Istanze	Tempo di Esecuzione [ms]		
	<i>Normale</i>	<i>Random</i>	<i>Invertito</i>
363	7	7	6

Passiamo ora a **Input_systems_long.txt**. In questo file è presente un solo sistema con 4 parametri i cui valori massimi possono essere modificati permettendoci così di creare un grande numero di sistemi. Fissandoci un tetto massimo di 100k sistemi, i dati raccolti sono i seguenti.

Tabella 2: Input_systems_long.txt

Istanze	Tempo di Esecuzione [ms]		
	<i>Normale</i>	<i>Random</i>	<i>Invertito</i>
1000	14	13	13
10000	131	125	128
100000	1394	1356	1400

Come è possibile osservare dalla tabella, la distanza tra gli ordinamenti cresce maggiore è la quantità di sistemi considerati. Il ritardo dell'ordinamento normale rispetto agli altri si accumula sistema dopo sistema finché non viene completamente distanziato.

Riportiamo ora tre esperimenti conclusivi in cui aumentiamo il numero di variabili ed equazioni. Eseguendo alcuni test su **Input_eq_25.txt**, il quale contiene 25 equazioni diverse con un totale di 25 variabili diverse, otteniamo un tempo di esecuzione pari a 5ms in ogni ordinamento.

Questa volta i risultati sono talmente piccoli da essere trascurabili.

Come da attesa, anche **Input_vars_70.txt** si comporta similmente: aumentare il numero di variabili ma non quello di equazioni non comporta un aumento degno di nota

nelle tempistiche di esecuzione (anche in questo parliamo di 1ms per ogni ordinamento) poiché la soluzione risulta triviale.

Concludiamo i test con **Input_vars_100.txt**, sistema che non solo aumenta le variabili utilizzate a 100, ma anche le equazioni (99).

Tabella 3: Input_vars_100.txt

Istanze	Tempo di Esecuzione [ms]		
	<i>Normale</i>	<i>Random</i>	<i>Invertito</i>
100	8	8	7

Nonostante la semplicità dell'input proposto, il grande numero di equazioni e variabili porta le tempistiche ad aumentare fino a una media di 8 ms ciascuna (risultato simile al primo test da noi eseguito, in cui risolvevamo un totale di 363 istanze).

In generale è stato possibile osservare che l'ordinamento più veloce risulta essere quello inverso, seguito dall'ordinamento normale e da quello randomico. Tuttavia, a causa dell'alta velocità di esecuzione di questa versione del progetto, il distacco tra i tre ordinamenti è molto meno marcato rispetto alla versione precedente. Risulta infatti difficile notare differenze sostanziali, l'unica si presenta quando raggiungiamo altissimi valori di sistemi (si veda Input_systems_long). Riguardando i test, le performance risultano essere molto buone e l'algoritmo capace di restituire un risultato in un tempo ristretto anche in casi con grandi quantità di sistemi o equazioni.

Conclusione

L'obiettivo del progetto è stato raggiunto, implementando una versione funzionante dell'algoritmo del simplesso generale per la soddisfacibilità. La grammatica, dapprima semplice, poco intuitiva e

contenuta, si è poi espansa e arricchita fino a giungere alla versione attuale. Inoltre grazie ai numerosi miglioramenti introdotti durante lo sviluppo dell'elaborato è stato possibile osservare un costante aumento delle performance rispetto alle prime versioni.

Riferimenti bibliografici

- [1] Daniel Kroening and Ofer Strichman. *Decision Procedures. An Algorithmic Point of View*. Springer, 2008.
- [2] Aaron R. Bradley and Zohar Manna. *The Calculus of Computation. Decision Procedures with Applications to Verification*. Springer, 2007.
- [3] Esercizi semplice geometria per fisica unipd. `eserciziSimplessoGeometriaX-FisicaUniPD.pdf`.